



2η Εργασία στο Μάθημα Ασφάλεια Συστημάτων

Περιγραφή

Η 2^η εργασία αποσκοπεί στην ανάπτυξη ενός ασφαλούς συστήματος αποθήκευσης αρχείων σε Python, με έμφαση σε πραγματικές τεχνικές κρυπτογραφίας και ελέγχου ταυτότητας. Θα υλοποιήσετε ένα CLI εργαλείο το οποίο ενσωματώνει:

- Εγγραφή και αυθεντικοποίηση χρηστών με ασφαλή αποθήκευση κωδικών
- Δημιουργία και χρήση ζεύγους κλειδιών RSA (public/private key)
- Ανέβασμα αρχείων με ψηφιακή υπογραφή και προστασία από replay attacks
- Κρυπτογράφηση αρχείων πριν την αποθήκευση (AES-256-GCM)
- Αποκρυπτογράφηση και ανάκτηση αρχείων από τον εξουσιοδοτημένο χρήστη
- Διαδραστικό Command-Line Interface (CLI)

Σημαντικές Οδηγίες

- Ο κώδικας πρέπει να τρέχει χωρίς σφάλματα με Python 3.9+
- Χρησιμοποιήστε μόνο τη βιβλιοθήκη cryptography για κρυπτογραφικές πράξεις
- Μην αλλάζετε τις έτοιμες συναρτήσεις (Μέρος Δ, show_menu, initialize_system)
- Μην συμπεριλαμβάνετε στην παράδοση: φακέλους keys/, secure_storage/, αρχεία
- Απαγορεύεται η χρήση έτοιμων βιβλιοθηκών που υλοποιούν ολόκληρες εργασίες (π.χ. flask_login)

Μέρος	Τίτλος	Περιγραφή
A	Hash & Salt (Εργ. 1)	Επαναχρησιμοποίηση υλοποίησης από 1η Εργασία
B	OTP & Εγγραφή	Εγγραφή χρήστη με μονόχρηστο OTP
Γ	Αυθεντικοποίηση	Σύνδεση χρήστη με έλεγχο password
Δ	Κλειδιά RSA	Δίνεται — Δημιουργία ζεύγους κλειδιών
E	Anti-Replay / Nonce	Προστασία από επανάληψη αιτημάτων



ΣΤ	Κρυπτογράφηση & Υπογραφή	AES-256-GCM + ψηφιακή υπογραφή RSA
Z	Upload / Download / CLI	Ολοκλήρωση pipeline και interface

2. Θεωρητικό Υπόβαθρο

2.1 OTP — One Time Password

Ένα OTP είναι κωδικός που ισχύει για μία και μόνο χρήση. Στην εφαρμογή μας:

- Ο διαχειριστής εκδίδει OTP για κάθε νέο χρήστη (π.χ. τον στέλνει μέσω email ή προφορικά)
- Ο χρήστης εισάγει το OTP κατά την εγγραφή — μετά δεν μπορεί να χρησιμοποιηθεί ξανά
- Αυτό εξασφαλίζει ότι μόνο εξουσιοδοτημένοι χρήστες μπορούν να δημιουργήσουν λογαριασμό

2.2 Hash & Salt (από Εργασία 1)

Χρησιμοποιείτε την υλοποίηση που έφτιαξες στην 1η Εργασία. Υπενθύμιση αρχών:

- Salt: τυχαία τιμή που συνδυάζεται με τον κωδικό πριν το hashing — αποτρέπει rainbow table attacks
- PBKDF2 / bcrypt / scrypt: συναρτήσεις με πολλαπλές επαναλήψεις — δυσκολεύουν brute force
- Αποθηκεύεται μόνο το hash+salt, ποτέ ο ίδιος ο κωδικός

2.3 RSA & Ψηφιακή Υπογραφή

- Κάθε χρήστης έχει ζεύγος κλειδιών: ιδιωτικό (για υπογραφή) και δημόσιο (για επαλήθευση)
- Η υπογραφή αποδεικνύει ότι το αρχείο ανέβηκε από τον συγκεκριμένο χρήστη (non-repudiation)
- Το nonce συμπεριλαμβάνεται στο υπογεγραμμένο payload — ακυρώνει replay attacks

2.4 AES-256-GCM

- Συμμετρική κρυπτογράφηση με authenticated encryption — ταυτόχρονα εμπιστευτικότητα και ακεραιότητα
- Κάθε αρχείο κρυπτογραφείται με νέο τυχαίο κλειδί και nonce — δεν επαναχρησιμοποιούνται



2.5 Replay Attack & Nonce

- Επίθεση: ο αντίπαλος υποκλέπτει ένα έγκυρο αίτημα (π.χ. upload) και το στέλνει ξανά
- Άμυνα: κάθε αίτημα φέρει μοναδικό nonce — αν χρησιμοποιηθεί ξανά, το σύστημα το απορρίπτει

3 Βήματα-Steps Υλοποίησης

3.1 Εγγραφή

1. Ο διαχειριστής εκτελεί `admin_issue_otp('alice')` → παράγεται OTP π.χ. A3F7C2B1
1. Ο χρήστης εισάγει: username, OTP, νέο password
2. Το σύστημα επαληθεύει το OTP (έγκυρο & αχρησιμοποίητο)
3. Το σύστημα καλεί `hash_password(password)` → αποθηκεύει hash+salt στο `users.json`
4. Δημιουργούνται αυτόματα τα κλειδιά RSA του χρήστη

3.2 Upload

5. Ο χρήστης επιλέγει αρχείο
6. Παράγεται nonce → συμπεριλαμβάνεται στο υπογεγραμμένο payload
7. Το nonce καταγράφεται ως χρησιμοποιημένο (anti-replay)
8. Το αρχείο κρυπτογραφείται με AES-256-GCM
9. Αποθηκεύεται: κρυπτογραφημένα bytes + metadata JSON (key, nonce, signature — base64)

4. Εκφώνηση ανά Μέρος

4.1 Μέρος A — Hash & Salt

Μαύρο κουτί: Οι συναρτήσεις `hash_password()` και `verify_password()` είναι η υλοποίησή σας από την 1η Εργασία. Αντιγράψτε τις ακριβώς στον κώδικα χωρίς να τις τροποποιήσετε. Ο starter κώδικας δεν τις παρέχει.

Απαιτούμενες υπογραφές:



```
hash_password(password: str, salt: str = None) -> tuple[str, str]
# Επιστρέφει (hash_hex, salt_hex)

verify_password(password: str, stored_hash: str, salt: str) -> bool
# Επιστρέφει True αν ο κωδικός είναι σωστός
```

Προϋπόθεση: Η υλοποίησή σας πρέπει να χρησιμοποιεί salt και αλγόριθμο με επαναλήψεις (PBKDF2, bcrypt ή scrypt). Απλό MD5/SHA1 χωρίς salt δεν γίνεται αποδεκτό.

4.2 Μέρος Β — OTP & Εγγραφή Χρήστη

Συμπληρώστε τη συνάρτηση `register_user(username, otp_input, password)`:

1. Φορτώστε το `OTP_DB` (`otp_store.json`)
2. Ελέγξτε αν υπάρχει εγγραφή για το `username` — αν όχι, τερματίστε με μήνυμα λάθους
3. Ελέγξτε αν το OTP έχει ήδη χρησιμοποιηθεί (`used == True`) — αν ναι, απορρίψτε
4. Συγκρίνετε `otp_input.upper()` με το αποθηκευμένο OTP
5. Αν είναι σωστό, σημειώστε `used = True` και αποθηκεύστε το `OTP_DB`
6. Ελέγξτε αν ο χρήστης υπάρχει ήδη στο `USERS_DB`
7. Καλέστε `hash_password(password)` → αποθηκεύστε `hash+salt+created_at` στο `USERS_DB`
8. Καλέστε `generate_user_keys(username)` για αυτόματη δημιουργία κλειδιών
9. Εμφανίστε επιβεβαίωση και επιστρέψτε `True`

Σημείωση OTP: Η συνάρτηση `admin_issue_otp()` δίνεται έτοιμη. Στο CLI, η επιλογή εγγραφής ζητά από τον χρήστη να εισάγει το OTP που του έχει δοθεί εκτός συστήματος (π.χ. email, τηλεφωνική επικοινωνία).

4.3 Μέρος Γ — Αυθεντικοποίηση

Συμπληρώστε τη συνάρτηση `authenticate_user(username, password)`:

10. Φορτώστε τη βάση χρηστών από `USERS_DB`
11. Ελέγξτε αν ο χρήστης υπάρχει
12. Ανακτήστε τα πεδία `password_hash` και `salt`
13. Καλέστε `verify_password(password, stored_hash, salt)` από Εργασία 1
14. Επιστρέψτε το αποτέλεσμα (`True/False`)



Ασφάλεια: Μην εμφανίζετε διαφορετικά μηνύματα για «χρήστης δεν υπάρχει» vs «λάθος κωδικός» — αυτό αποκαλύπτει πληροφορίες σε επιτιθέμενο. Χρησιμοποιήστε πάντα το ίδιο μήνυμα: «Λάθος στοιχεία σύνδεσης».

4.4 Μέρος Δ — Κλειδιά RSA

Δίνεται: Οι συναρτήσεις `generate_user_keys()`, `load_private_key()`, `load_public_key()` παρέχονται έτοιμες στον `starter` κώδικα. Καλούνται αυτόματα κατά την εγγραφή. Δεν χρειάζεται να τις τροποποιήσετε.

Κατανοήστε τι κάνουν:

- `generate_user_keys()`: δημιουργεί RSA-2048 ζεύγος και το αποθηκεύει σε PEM αρχεία στον φάκελο `keys/`
- `load_private_key(username)`: φορτώνει το ιδιωτικό κλειδί για υπογραφή
- `load_public_key(username)`: φορτώνει το δημόσιο κλειδί για επαλήθευση

4.5 Μέρος Ε — Anti-Replay / Nonce

Συμπληρώστε τη συνάρτηση `is_nonce_valid(nonce)`:

15. Φορτώστε τη λίστα `nonces` από το `NONCE_DB`
16. Αν το `nonce` βρίσκεται ήδη στη λίστα → εκτυπώστε προειδοποίηση `Replay Attack` και επιστρέψτε `False`
17. Αν είναι καινούριο → προσθέστε το στη λίστα, αποθηκεύστε το αρχείο, επιστρέψτε `True`

Bonus (προαιρετικό): Αποθηκεύστε και `timestamp` μαζί με κάθε `nonce`. Κατά τον έλεγχο, καθαρίστε `nonces` παλαιότερα των 24 ωρών ώστε το αρχείο να μην μεγαλώνει απεριόριστα.

4.6 Μέρος ΣΤ — Κρυπτογράφηση & Ψηφιακή Υπογραφή

ΣΤ(α) — Κρυπτογράφηση AES-256-GCM

Συμπληρώστε τις `encrypt_file()` και `decrypt_file()`:

encrypt_file(data: bytes) → tuple:

18. Δημιουργήστε τυχαίο κλειδί 32 bytes: `key = os.urandom(32)`
19. Δημιουργήστε τυχαίο nonce 12 bytes: `nonce_aes = os.urandom(12)`
20. Κρυπτογραφήστε: `aesgcm = AESGCM(key) → ciphertext = aesgcm.encrypt(nonce_aes, data, None)`
21. Επιστρέψτε (`ciphertext`, `key`, `nonce_aes`)



decrypt_file(ciphertext, key, nonce_aes) → bytes:

22. Δημιουργήστε AESGCM(key)
23. Επιστρέψτε aesgcm.decrypt(nonce_aes, ciphertext, None)

ΣΤ(β) — Ψηφιακή Υπογραφή RSA-PSS

Συμπληρώστε τις sign_data() και verify_signature():

sign_data(username, data, nonce) → bytes:

24. Φορτώστε ιδιωτικό κλειδί: private_key = load_private_key(username)
25. Δημιουργήστε payload: payload = data + nonce.encode()
26. Υπογράψτε με RSA-PSS + SHA256 και επιστρέψτε την υπογραφή

verify_signature(username, data, nonce, sig) → bool:

27. Φορτώστε δημόσιο κλειδί: public_key = load_public_key(username)
28. Δημιουργήστε ίδιο payload: payload = data + nonce.encode()
29. Καλέστε public_key.verify() μέσα σε try/except — επιστρέψτε True/False

RSA-PSS Template: private_key.sign(payload, padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH), hashes.SHA256()) — public_key.verify(sig, payload, ίδιο padding, hashes.SHA256()) πετά InvalidSignature αν αποτύχει.

4.7 Μέρος Ζ — Upload, Download & CLI

Ζ(α) — upload_file(username, filepath)

Ακολουθήστε αυτή τη ροή:

30. Διαβάστε τα bytes: data = Path(filepath).read_bytes()
31. Παράξτε nonce: nonce = generate_nonce()
32. Υπογράψτε: signature = sign_data(username, data, nonce)
33. Ελέγξτε nonce: αν is_nonce_valid(nonce) == False → σταματήστε
34. Κρυπτογραφήστε: ciphertext, key, nonce_aes = encrypt_file(data)
35. Αποθηκεύστε κρυπτογραφημένα bytes στο STORAGE_DIR/<filename>.enc
36. Αποθηκεύστε metadata JSON: signature, key, nonce_aes (όλα base64), nonce, uploader, original_filename, timestamp

Ζ(β) — download_file(username, filename)

37. Φορτώστε metadata JSON από STORAGE_DIR/<filename>.json
38. Διαβάστε κρυπτογραφημένα bytes από .enc αρχείο



39. Αποκωδικοποιήστε base64: key, nonce_aes, signature
40. Αποκρυπτογραφήστε: plaintext = decrypt_file(ciphertext, key, nonce_aes)
41. Επαληθεύστε υπογραφή: verify_signature() — εμφανίστε προειδοποίηση αν αποτύχει
42. Αποθηκεύστε τοπικά ως downloaded_<original_filename>

Ζ(γ) — main() — CLI Loop

43. Καλέστε initialize_system() στην αρχή
44. Διατηρήστε μεταβλητή logged_in_user = None για το session
45. Επιλογή 1 (Εγγραφή): ζητήστε username, OTP, password → καλέστε register_user()
46. Επιλογή 2 (Σύνδεση): ζητήστε username, password → καλέστε authenticate_user() → ενημερώστε session
47. Επιλογή 3 (Upload): απαιτεί ενεργό session → ζητήστε filepath → upload_file()
48. Επιλογή 4 (Download): απαιτεί ενεργό session → ζητήστε filename → download_file()
49. Επιλογή 0: εκτύπωση αποχαιρετισμού και έξοδος

Σημείωση admin: Για να δοκιμάσετε την εγγραφή, μπορείτε να καλέσετε admin_issue_otp('username') απευθείας στο CLI (π.χ. επιλογή 9 «Admin: Έκδοση OTP») ή να το καλέíte manually πριν το main loop κατά το testing.

5. Starter Code

Αποθηκεύστε τον παρακάτω κώδικα σε αρχείο secure_storage.py. Συμπληρώστε μόνο τα σημεία # TODO:

```
#!/usr/bin/env python3
"""
2η Εργασία — Ασφάλεια Συστημάτων
Secure File Storage System

ΟΔΗΓΙΕΣ:
• Συμπληρώστε ΜΟΝΟ τα τμήματα # TODO
• Για hash_password() και verify_password() χρησιμοποιήστε
  ΤΗ ΔΙΚΗ ΣΑΣ υλοποίηση από την Εργασία 1
• Μην τροποποιείτε τις υπόλοιπες συναρτήσεις
"""

import os, json, secrets, base64
from pathlib import Path
from datetime import datetime
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ
ΣΧΟΛΗ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
& ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

```
from cryptography.hazmat.backends import default_backend
```

```
# -----  
# ΣΤΑΘΕΡΕΣ  
# -----  
USERS_DB = Path('users.json')  
STORAGE_DIR = Path('secure_storage')  
KEYS_DIR = Path('keys')  
NONCE_DB = Path('nonces.json')  
OTP_DB = Path('otp_store.json') # Δίνεται από διαχειριστή  
  
def initialize_system():  
    STORAGE_DIR.mkdir(exist_ok=True)  
    KEYS_DIR.mkdir(exist_ok=True)  
    for f, default in [(USERS_DB, {}), (NONCE_DB, []), (OTP_DB, {})]:  
        if not f.exists():  
            f.write_text(json.dumps(default))  
  
# -----  
# ΜΕΡΟΣ Α – Hash & Salt (από Εργασία 1 – ΜΑΥΡΟ ΚΟΥΤΙ)  
# -----  
# Αντιγράψτε εδώ τις δύο συναρτήσεις σας από την Εργασία 1:  
#  
# hash_password(password: str, salt: str = None) -> tuple[str, str]  
#     Επιστρέφει (hash_hex, salt_hex)  
#  
# verify_password(password: str, stored_hash: str, salt: str) -> bool  
#     Επιστρέφει True αν ο κωδικός είναι σωστός  
#  
# TODO – ΜΕΡΟΣ Α: Επικολλήστε τις υλοποιήσεις σας από Εργασία 1  
  
# -----  
# ΜΕΡΟΣ Β – OTP & Εγγραφή Χρήστη (Δίνεται μερικώς)  
# -----  
  
def admin_issue_otp(username: str) -> str:  
    """Διαχειριστής: δημιουργεί και αποθηκεύει OTP για username."""  
    otp = secrets.token_hex(4).upper() # 8-ψήφιο hex OTP  
    store = json.loads(OTP_DB.read_text())  
    store[username] = {  
        "otp": otp,  
        "issued_at": datetime.now().isoformat(),  
        "used": False  
    }  
    OTP_DB.write_text(json.dumps(store, indent=2))  
    print(f"[ADMIN] OTP για '{username}': {otp}")  
    return otp  
  
def register_user(username: str, otp_input: str, password: str) -> bool:  
    """  
    TODO – ΜΕΡΟΣ Β:  
    Εγγραφή χρήστη με επαλήθευση OTP.  
    Ροή:  
    1. Φορτώστε το OTP_DB και ελέγξτε αν υπάρχει OTP για το username  
    2. Ελέγξτε αν το OTP δεν έχει χρησιμοποιηθεί ξανά (used == False)  
    3. Συγκρίνετε το otp_input με το αποθηκευμένο (case-insensitive)  
    4. Αν είναι σωστό: σημειώστε το ως used=True  
    5. Ελέγξτε αν ο χρήστης υπάρχει ήδη στο USERS_DB  
    6. Δημιουργήστε hash+salt του password (από Εργασία 1)  
    """
```



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ
ΣΧΟΛΗ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
& ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

```
7. Αποθηκεύστε χρήστη στο USERS_DB
8. Καλέστε generate_user_keys(username)
9. Επιστρέψτε True αν όλα πήγαν καλά
"""

# TODO - ΜΕΡΟΣ Β
pass

# -----
# ΜΕΡΟΣ Γ - Αυθεντικοποίηση (Δίνεται δομή)
# -----

def authenticate_user(username: str, password: str) -> bool:
    """
    TODO - ΜΕΡΟΣ Γ:
    Ελέγξτε τα στοιχεία σύνδεσης.
    1. Φορτώστε USERS_DB
    2. Ελέγξτε αν ο χρήστης υπάρχει
    3. Ανακτήστε stored_hash και salt
    4. Καλέστε verify_password() από Εργασία 1
    5. Επιστρέψτε True/False
    """
    # TODO - ΜΕΡΟΣ Γ
    pass

# -----
# ΜΕΡΟΣ Δ - Κλειδιά RSA (Δίνεται)
# -----

def generate_user_keys(username: str):
    """Δημιουργία RSA-2048 ζεύγους κλειδών."""
    pk = rsa.generate_private_key(65537, 2048, default_backend())
    (KEYS_DIR / f"{username}_private.pem").write_bytes(
        pk.private_bytes(serialization.Encoding.PEM,
                        serialization.PrivateFormat.TraditionalOpenSSL,
                        serialization.NoEncryption()))
    (KEYS_DIR / f"{username}_public.pem").write_bytes(
        pk.public_key().public_bytes(serialization.Encoding.PEM,
                                    serialization.PublicFormat.SubjectPublicKeyInfo))

def load_private_key(username: str):
    return serialization.load_pem_private_key(
        (KEYS_DIR / f"{username}_private.pem").read_bytes(),
        password=None, backend=default_backend())

def load_public_key(username: str):
    return serialization.load_pem_public_key(
        (KEYS_DIR / f"{username}_public.pem").read_bytes(),
        backend=default_backend())

# -----
# ΜΕΡΟΣ Ε - Nonce / Anti-Replay (Δίνεται δομή)
# -----

def generate_nonce() -> str:
    return secrets.token_hex(16)

def is_nonce_valid nonce: str) -> bool:
    """
    TODO - ΜΕΡΟΣ Ε:
```



ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ
ΣΧΟΛΗ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ
& ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

Ελέγξτε αν το nonce είναι μοναδικό.

1. Φορτώστε τη λίστα από NONCE_DB
2. Αν υπάρχει ήδη → return False (Replay Attack)
3. Αλλιώς → αποθηκεύστε το, return True

"""

TODO — ΜΕΡΟΣ Ε

pass

#

ΜΕΡΟΣ ΣΤ — Κρυπτογράφηση & Υπογραφή (Δίνεται δομή)

#

def encrypt_file(data: bytes) -> tuple:

"""

TODO — ΜΕΡΟΣ ΣΤ (α):

Κρυπτογράφηση με AES-256-GCM.

Επιστρέψτε: (ciphertext, key_bytes, nonce_aes_bytes)

"""

TODO — ΜΕΡΟΣ ΣΤ (α)

pass

def decrypt_file(ciphertext: bytes, key: bytes, nonce_aes: bytes) -> bytes:

"""

TODO — ΜΕΡΟΣ ΣΤ (α):

Αποκρυπτογράφηση AES-256-GCM.

"""

TODO — ΜΕΡΟΣ ΣΤ (α)

pass

def sign_data(username: str, data: bytes, nonce: str) -> bytes:

"""

TODO — ΜΕΡΟΣ ΣΤ (β):

Ψηφιακή υπογραφή RSA-PSS+SHA256.

payload = data + nonce.encode()

"""

TODO — ΜΕΡΟΣ ΣΤ (β)

pass

def verify_signature(username: str, data: bytes, nonce: str, sig: bytes) -> bool:

"""

TODO — ΜΕΡΟΣ ΣΤ (β):

Επαλήθευση ψηφιακής υπογραφής.

Επιστρέψτε True/False.

"""

TODO — ΜΕΡΟΣ ΣΤ (β)

pass

#

ΜΕΡΟΣ Ζ — Upload / Download / CLI (Δίνεται δομή)

#

def upload_file(username: str, filepath: str):

"""

TODO — ΜΕΡΟΣ Ζ (α):

Ανέβασμα αρχείου με ασφάλεια.

Ροή: read → nonce → sign → validate_nonce → encrypt → save

"""

TODO — ΜΕΡΟΣ Ζ (α)

pass



```
def download_file(username: str, filename: str):
    """
    TODO - ΜΕΡΟΣ Ζ (β):
    Λήψη και αποκρυπτογράφηση αρχείου.
    Ροή: load metadata → decrypt → verify_signature → save locally
    """
    # TODO - ΜΕΡΟΣ Ζ (β)
    pass

def show_menu():
    print("\n" + "="*48)
    print("    Secure File Storage - Εργασία 2")
    print("="*48)
    print("    1. Εγγραφή (με OTP)")
    print("    2. Σύνδεση")
    print("    3. Ανέβασμα αρχείου")
    print("    4. Κατέβασμα αρχείου")
    print("    0. Έξοδος")
    print("="*48)

def main():
    """
    TODO - ΜΕΡΟΣ Ζ (γ):
    Κύριο CLI loop.
    - Αρχικοποίηση συστήματος
    - Loop: εμφάνιση μενού, ανάγνωση επιλογής
    - Διαχείριση session (logged_in_user = None)
    - Οι επιλογές 3 & 4 απαιτούν ενεργό session
    """
    initialize_system()
    logged_in_user = None
    # TODO - ΜΕΡΟΣ Ζ (γ)
    pass

if __name__ == "__main__":
    main()
```

6. Εγκατάσταση & Εκτέλεση

Απαιτήσεις

- Python 3.9 ή νεότερο
- `pip install cryptography`

Εκτέλεση

```
python secure_storage.py
```

Δοκιμή βήμα-βήμα

50. Εκτελέστε το πρόγραμμα
51. Επιλέξτε «Admin: Έκδοση OTP» (αν το υλοποιήσατε στο CLI) ή καλέστε `admin_issue_otp()` χειροκίνητα



52. Επιλέξτε «Εγγραφή» → εισάγετε username, OTP, password
53. Επιλέξτε «Σύνδεση» → εισάγετε credentials
54. Επιλέξτε «Ανέβασμα» → εισάγετε path αρχείου (π.χ. test.txt)
55. Ελέγξτε ότι δημιουργήθηκαν τα αρχεία .enc και .json στο secure_storage/
56. Επιλέξτε «Κατέβασμα» → εισάγετε filename → επαληθεύστε ότι το αρχείο ανακτήθηκε σωστά

7. Παράδοση & Αξιολόγηση

Τι παραδίδετε

- secure_storage.py — πλήρης κώδικας
- report.pdf (1–2 σελίδες): σύντομη τεκμηρίωση με περιγραφή κάθε μέρους
- Screenshots που αποδεικνύουν: εγγραφή με OTP, σύνδεση, upload, download, απόρριψη replay